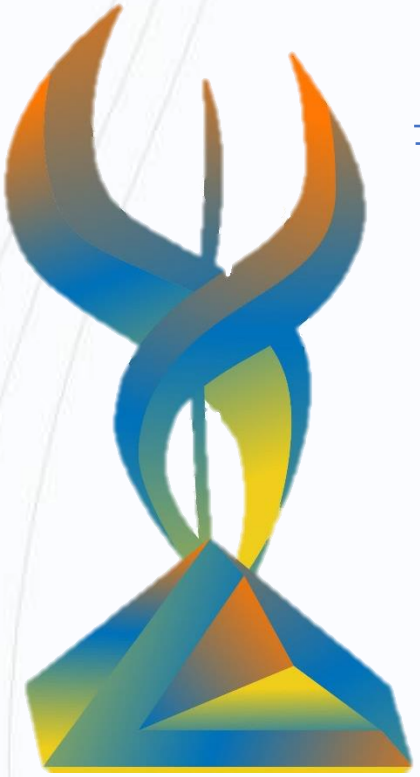
Republic of the Philippines
Bicol University
POLANGUI CAMPUS
Polangui, Albay

P2: FRONTEND DEVELOPMENT BASED ON PROJECT DESIGN

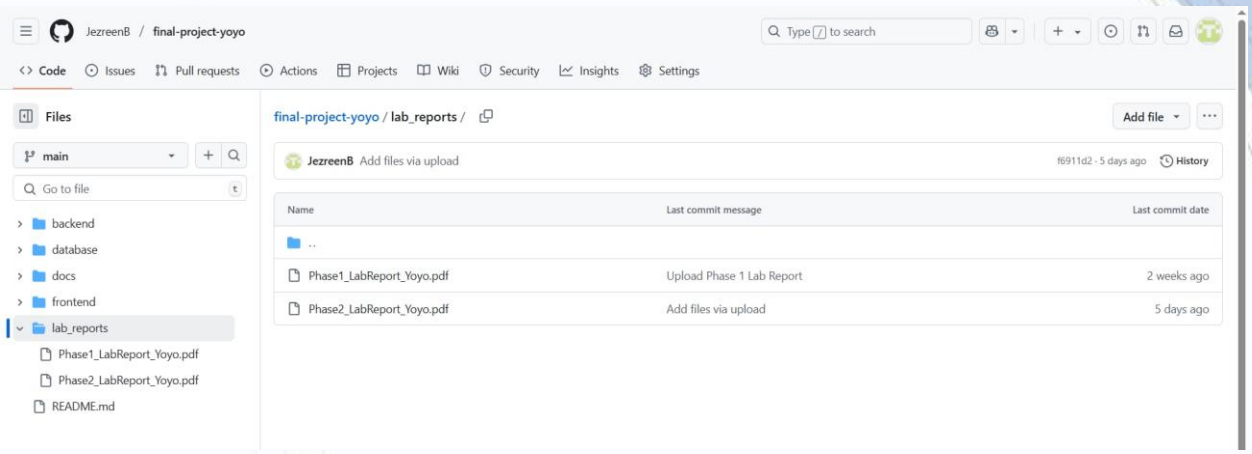


GROUP YOYO

Llona, Yasmien Mae M.
Lusantas, Maryan D.
Ruiz, Catlyn L.
Fermin, Yasmin M.
Bron, Niña Jezreen M.
Idanan, John Mark C.

INITIAL LAB REPORT FOR PHASE 2 UPLOADED IN GITHUB

Content: All Working frontend pages with annotations



OVERVIEW OF BACKEND SETUP

1. **Technology Stack**

- Node.js with Express framework for the server.
- MongoDB with Mongoose ODM for database operations.
- JSON Web Tokens (JWT) for authentication.
- bcrypt for password hashing.
- Middleware for authentication and request parsing.

2. **Project Structure**

- server.js: Entry point of the backend server. Sets up Express app, connects to MongoDB, configures middleware (CORS, body-parser), and mounts route handlers.
- config/db.js: Handles MongoDB connection using Mongoose.
- routes/: Contains route definitions for different resources:
    - userRoutes.js: User registration and login.
    - productRoutes.js: Product listing and adding products.
    - reviewRoutes.js: Submitting and fetching product reviews.
    - cartRoutes.js: Managing shopping cart items.
    - orderRoutes.js: Placing orders and order management.
- controllers/: Contains controller logic for each resource, handling business logic and database interactions.
- models/: Mongoose schemas and models for User, Product, Review, CartItem, and Order.

- middleware/authenticateToken.js: Middleware to verify JWT tokens and protect routes.
- package.json: Lists dependencies and scripts for running the server.

3. Authentication

- Users register and login via /api/users routes.
- JWT tokens are issued on login and required for protected routes (adding products, managing cart, placing orders, submitting reviews).
- authenticateToken middleware verifies tokens on protected routes.

4. API Endpoints:

- /api/users: Register and login.
- /api/products: Get products, add product (admin only).
- /api/reviews: Submit and get reviews.
- /api/cart: Get, add, remove cart items.
- /api/orders: Place orders, get orders, cancel orders.

5. Security:

- Passwords are hashed with bcrypt.
- Role-based access control is implied (e.g., admin role for adding products).
- JWT tokens secure protected endpoints.

CODE SNIPPETS OF SERVER.JS, SAMPLE MODEL, AND ROUTES

SERVER.JS

backend > JS server.js > ...

```
require('dotenv').config();

const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const connectDB = require('./config/db');
const userRoutes = require('./routes/userRoutes');

const app = express();
const PORT = process.env.PORT || 3001;

connectDB();

app.use(cors());
app.use(bodyParser.json());

app.use('/api/users', userRoutes);

const productRoutes = require('./routes/productRoutes');
const reviewRoutes = require('./routes/reviewRoutes');
const cartRoutes = require('./routes/cartRoutes');
const orderRoutes = require('./routes/orderRoutes');

app.use('/api/products', productRoutes);
app.use('/api/reviews', reviewRoutes);
app.use('/api/cart', cartRoutes);
app.use('/api/orders', orderRoutes);

app.listen(PORT, () => {
  console.log("Server running on http://localhost:" + PORT);
});

```

SAMPLE MODELS

backend > models > JS Product.js > ...

```
1   const mongoose = require('mongoose');
2
3   const productSchema = new mongoose.Schema({
4     name: { type: String, required: true },
5     category: String,
6     price: { type: Number, required: true },
7     image: String,
8   });
9
10  module.exports = mongoose.model('Product', productSchema);
11
```

backend > models > JS User.js > ...

```
1   const mongoose = require('mongoose');
2
3   const userSchema = new mongoose.Schema({
4     fullName: { type: String, required: true },
5     address: { type: String, required: true },
6     email: { type: String, required: true, unique: true },
7     password: { type: String, required: true },
8     role: { type: String, enum: ['user', 'admin'], default: 'user' }
9   });
10
11  module.exports = mongoose.model('User', userSchema);
12
```

backend > models > JS Order.js > ...

```
1   const mongoose = require('mongoose');
2
3   const orderItemSchema = new mongoose.Schema({
4     productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
5     quantity: { type: Number, required: true },
6     price: { type: Number, required: true },
7   });
8
9   const orderSchema = new mongoose.Schema({
10    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
11    orderNumber: { type: String, required: true, unique: true },
12    status: { type: String, required: true },
13    total: { type: Number, required: true },
14    deliveryAddress: { type: String, required: true },
15    paymentMethod: { type: String, required: true },
16    shippingMethod: { type: String, required: true },
17    orderDate: { type: Date, default: Date.now },
18    items: [orderItemSchema],
19  });
20
21  module.exports = mongoose.model('Order', orderSchema);
22
```

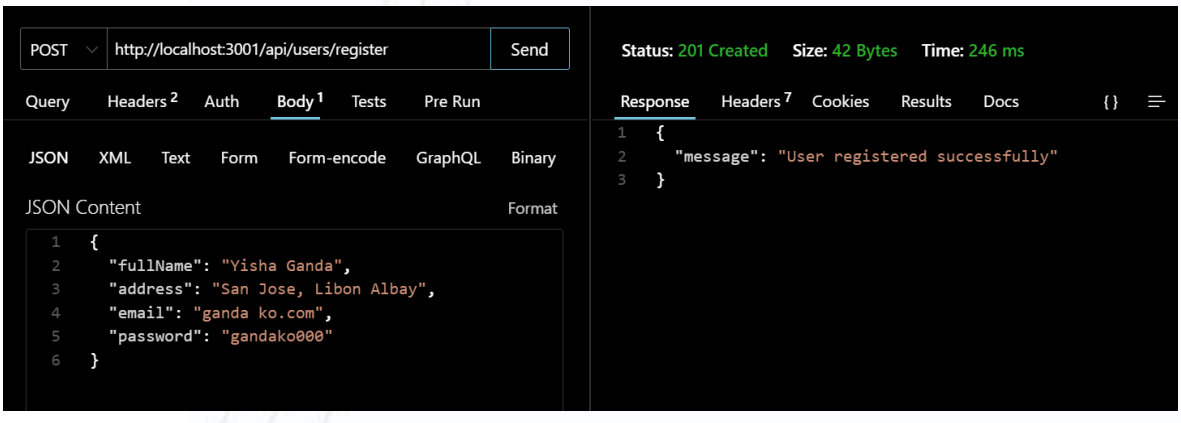
backend > models > JS Review.js > ...

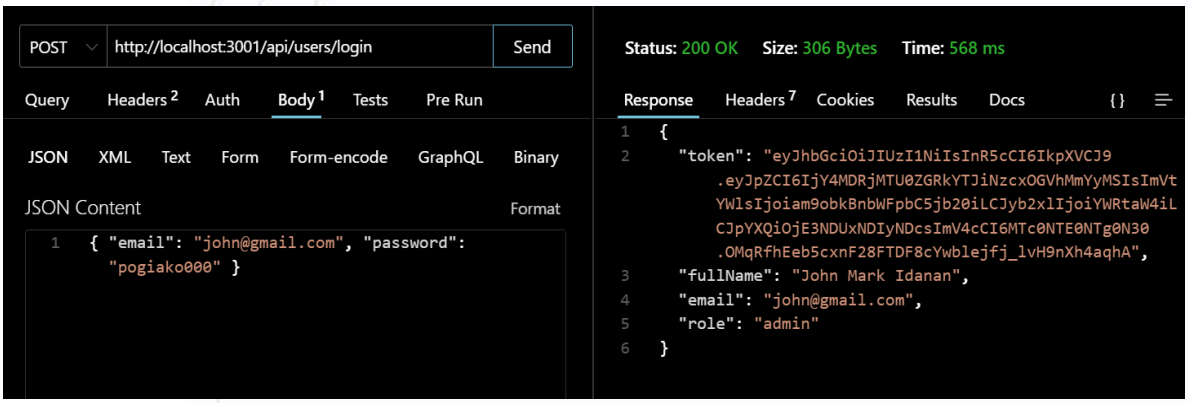
```js
const mongoose = require('mongoose');

const reviewSchema = new mongoose.Schema({
  productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  rating: { type: Number, required: true },
  text: String,
  date: { type: Date, default: Date.now },
});

module.exports = mongoose.model('Review', reviewSchema);

```

backend > models > JS CartItem.js > ...

```js
const mongoose = require('mongoose');

const cartItemSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  quantity: { type: Number, required: true },
});

module.exports = mongoose.model('CartItem', cartItemSchema);

```

API TESTING SCREENSHOTS





POST http://localhost:3001/api/products Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
"name": "NZXT H510 Elite",
"category": "PC Case",
"price": 89.99,
"image": "images/gc1.jpg"
}
```

Status: 201 Created Size: 27 Bytes Time: 71 ms

Response Headers 7 Cookies Results Docs { }

```
{
  "message": "Product added"
}
```

POST http://localhost:3001/api/orders Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
"deliveryAddress": "San Jose, Libon ALbay",
"paymentMethod": "Credit Card",
"shippingMethod": "Standard",
"items": [
{
"productId": "6804c242ddda2b7718ea2f25",
"price": 89.99,
"quantity": 2
}
]
}
```

Status: 201 Created Size: 52 Bytes Time: 5.97 s

Response Headers 7 Cookies Results Docs { }

```
{
  "message": "Order placed",
  "orderNumber": "NG-469740"
}
```

POST http://localhost:3001/api/cart Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

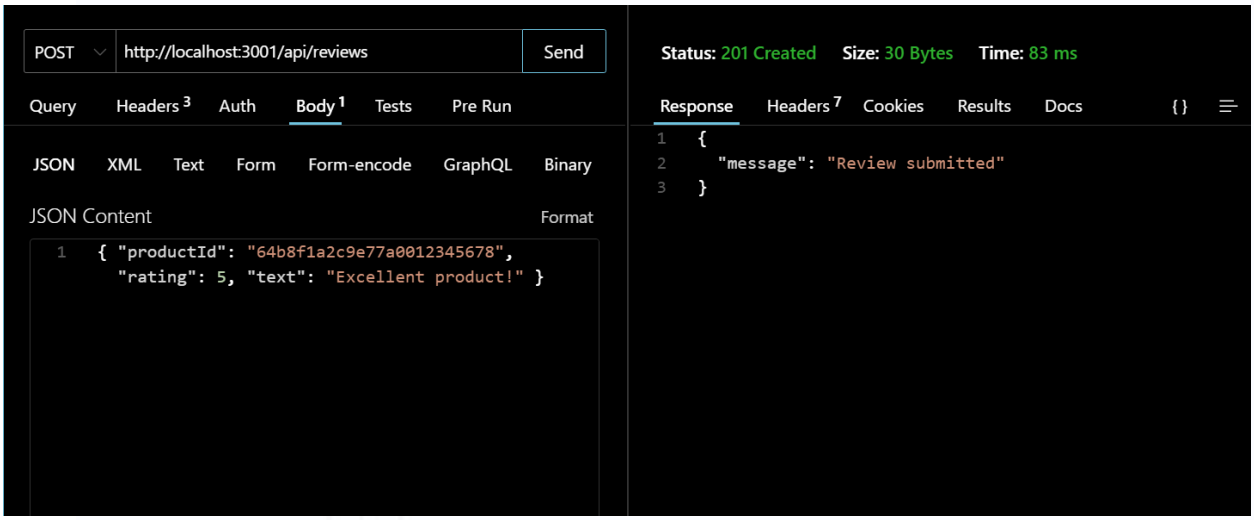
JSON Content Format

```
{
"productId": "64b8f1a2c9e77a0012345678",
"quantity": 2
}
```

Status: 200 OK Size: 26 Bytes Time: 111 ms

Response Headers 7 Cookies Results Docs { }

```
{
  "message": "Cart updated"
}
```


POST
http://localhost:3001/api/reviews
Send
Query
Headers 3
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
1 { "productId": "64b8f1a2c9e77a0012345678",
"rating": 5, "text": "Excellent product!" }
Status: 201 Created
Size: 30 Bytes
Time: 83 ms
Response
Headers 7
Cookies
Results
Docs
{ }
1 {
2 "message": "Review submitted"
3 }